

Personal Portfolio Website - Internship Task Submission

Project Title

Personal Portfolio Website

Objective

Build a full-stack personal portfolio website to showcase projects, skills, and contact information using frontend, backend, database, and deployment technologies.

Technologies Used

Frontend

- HTML
- CSS
- JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB

Deployment

- Netlify / Vercel
-

Project Description

This project is a responsive full-stack personal portfolio website developed as part of an internship task. The website displays personal information, technical skills, project details, and includes a contact form connected to a backend server.

The application demonstrates:

- Frontend development

- Backend API creation
 - Database integration
 - Deployment process
-

Full-Stack Personal Portfolio Website

Project Overview

Build a modern full-stack personal portfolio website to showcase:

- About Me
- Skills
- Projects
- Resume
- Contact Form
- Social Media Links

Tech Stack:

- Frontend: React.js + Tailwind CSS
 - Backend: Node.js + Express.js
 - Database: MongoDB
 - Deployment: Vercel (Frontend) + Render (Backend)
-

Folder Structure

```
portfolio-project/
├── client/                # Frontend
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── App.js
│   │   └── main.jsx
│   └── package.json
├── server/               # Backend
│   ├── models/
│   ├── routes/
│   ├── server.js
│   └── package.json
└── README.md
```

Step 1: Frontend Setup

Create React App

```
npm create vite@latest client  
cd client  
npm install
```

Install Tailwind CSS

```
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

Configure Tailwind

tailwind.config.js

```
export default {  
  content: [  
    "./index.html",  
    "./src/**/*..{js,ts,jsx,tsx}",  
  ],  
  theme: {  
    extend: {},  
  },  
  plugins: [],  
}
```

index.css

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

Step 2: App UI

App.jsx

```
import React from 'react';  
  
function App() {
```

```

return (
  <div className="bg-gray-900 text-white min-h-screen">

    {/* Navbar */}
    <nav className="flex justify-between items-center p-5 bg-black">
      <h1 className="text-2xl font-bold">My Portfolio</h1>
      <ul className="flex gap-6">
        <li>Home</li>
        <li>Projects</li>
        <li>Skills</li>
        <li>Contact</li>
      </ul>
    </nav>

    {/* Hero Section */}
    <section className="text-center py-20">
      <h2 className="text-5xl font-bold">Hi, I'm Shwetha</h2>
      <p className="mt-4 text-xl text-gray-300">
        Full Stack Developer | Java Programmer
      </p>
    </section>

    {/* Skills */}
    <section className="p-10">
      <h2 className="text-3xl font-bold mb-5">Skills</h2>

      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        <div className="bg-gray-800 p-4 rounded">Java</div>
        <div className="bg-gray-800 p-4 rounded">React</div>
        <div className="bg-gray-800 p-4 rounded">Node.js</div>
        <div className="bg-gray-800 p-4 rounded">MongoDB</div>
      </div>
    </section>

    {/* Projects */}
    <section className="p-10">
      <h2 className="text-3xl font-bold mb-5">Projects</h2>

      <div className="grid md:grid-cols-3 gap-6">
        <div className="bg-gray-800 p-5 rounded">
          <h3 className="text-xl font-bold">Portfolio Website</h3>
          <p className="mt-2 text-gray-300">
            Personal portfolio using React and Node.js.
          </p>
        </div>

        <div className="bg-gray-800 p-5 rounded">
          <h3 className="text-xl font-bold">Student Management</h3>
          <p className="mt-2 text-gray-300">
            CRUD application using MongoDB.
          </p>
        </div>
      </div>
    </section>
  </div>
)

```

```

        </div>
      </div>
    </section>

    { /* Contact */ }
    <section className="p-10">
      <h2 className="text-3xl font-bold mb-5">Contact</h2>

      <form className="flex flex-col gap-4 max-w-md">
        <input
          type="text"
          placeholder="Your Name"
          className="p-3 rounded bg-gray-800"
        />

        <input
          type="email"
          placeholder="Your Email"
          className="p-3 rounded bg-gray-800"
        />

        <textarea
          placeholder="Message"
          className="p-3 rounded bg-gray-800"
        ></textarea>

        <button className="bg-blue-600 p-3 rounded">
          Send Message
        </button>
      </form>
    </section>
  </div>
);
}

export default App;

```

Step 3: Backend Setup

Create Server

```

mkdir server
cd server
npm init -y

```

Install Packages

```
npm install express mongoose cors dotenv nodemon
```

Step 4: Backend Code

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.log(err));

app.get('/', (req, res) => {
  res.send('Portfolio Backend Running');
});

app.post('/contact', (req, res) => {
  const { name, email, message } = req.body;

  console.log(name, email, message);

  res.json({ message: 'Message received!' });
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 5: Environment Variables

.env

```
MONGO_URI=your_mongodb_connection_string  
PORT=5000
```

Step 6: Connect Frontend to Backend

Contact API Example

```
const sendMessage = async () => {  
  const response = await fetch('http://localhost:5000/contact', {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify({  
      name,  
      email,  
      message  
    })  
  });  
  
  const data = await response.json();  
  alert(data.message);  
}
```

Step 7: Deployment

Frontend Deployment

Use:

- Vercel
- Netlify

Backend Deployment

Use:

- Render

- Railway

Database

Use:

- MongoDB Atlas
-

Features to Add

- Dark Mode
 - Project Images
 - Resume Download Button
 - Animations using Framer Motion
 - GitHub API Integration
 - Admin Dashboard
 - Blog Section
 - Visitor Counter
-

Sample Portfolio Sections

Home

- Introduction
- Profile Photo
- Social Media Icons

About

- Education
- Internship Experience
- Career Goals

Skills

- Java
- HTML
- CSS
- JavaScript
- React
- SQL

Projects

- LeetCode Tracker
- Student Management System

- E-commerce Website

Contact

- Email Form
 - LinkedIn
 - GitHub
-

Commands to Run Project

Frontend

```
cd client  
npm run dev
```

Backend

```
cd server  
nodemon server.js
```

Final Result

You will have a complete full-stack portfolio website with:

- Responsive Design
- Backend API
- Database Connection
- Contact Form
- Project Showcase
- Professional UI

This project is perfect for:

- Internship Applications
- LinkedIn Showcase
- Resume Projects
- Placement Portfolio